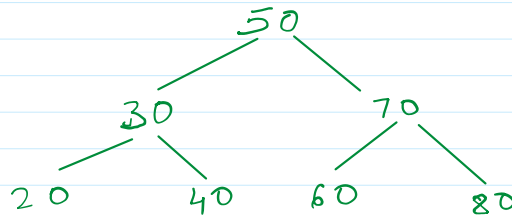


Binary Search Tree (BST)

11 June 2026 10:06

A **Binary Search Tree (BST)** is a special type of tree in which:

- Every node has at most two children (0, 1 or 2)
- The left subtree contains values smaller than the parent node.
- The right subtree contains values greater or equal to the parent node.
- Both left and right subtrees are themselves BST.



BST property: For every node.
Left child $<$ Parent $\leq$ Right child.

Why use binary search trees?

BST provides efficient operations for:

- Insertion
- Searching
- Deletion



'n' $\leftarrow$ we call it linear
 $\uparrow$
 $O(n)$

Disadvantage $\rightarrow$
(insertion of one element takes linear time).

$\uparrow$ to make this cell empty, I need to shift 'n' elements.
 $\uparrow$ Shifting is costly in Array.
 $\uparrow$ why?
we need to shift 'n' elements.

Advantage of array: If you know the index, you can access the element directly.

$n^2 \leftarrow$ quadratic $\rightarrow O(n^2)$
 $n^3 \leftarrow$ cubics $\rightarrow O(n^3)$

$\rightarrow$ I want constant time for insertion. $O(1)$

Linked List.

$\rightarrow$ Insertion
 $\rightarrow$ Deletion

Shifting of elements not needed.

$\uparrow$ In Linked list, search is inefficient.

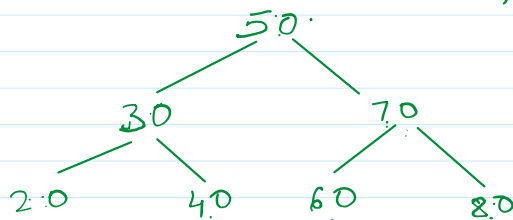
Disadvantage $\rightarrow$ is linear

Disadvantage $\rightarrow$ is linear $\rightarrow$ ∞
 To make our search efficient (BST).

Insertion
 Deletion
 and search } are efficient. (It uses binary search logic).

BST always maintains order.

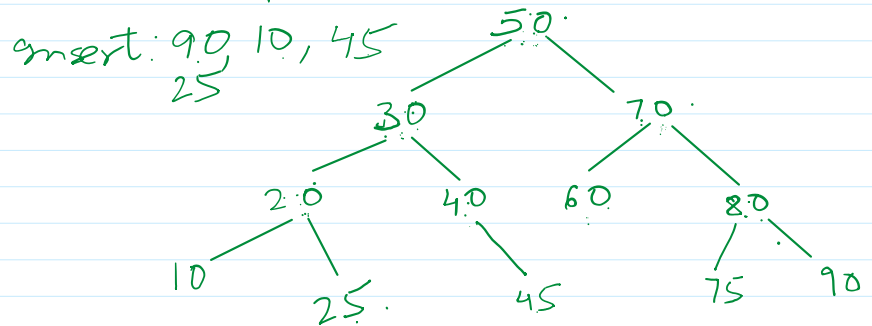
In-order traversal of BST you will get sorted data or data in order.



In order $\rightarrow L \rightarrow R \rightarrow R = 20, 30, 40, 50, 60, 70, 80$

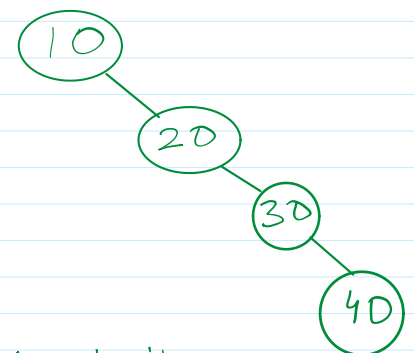
Time complexity (Average Case)

Operation	Complexity
insert	$O(\log n)$
delete	$O(\log n)$
search	$O(\log n)$



```

package BinaryTree;
public class Node {
    private int data;
    private Node left, right;
    public Node(){
        data = 0;
        left = right = null;
    }
    public Node(int data) {
        this.data = data;
    }
    public int getData() {
        return data;
    }
    public void setData(int data) {
        this.data = data;
    }
    public Node getLeft() {
        return left;
    }
    public void setLeft(Node left) {
        this.left = left;
    }
  }
  
```



Complexity becomes: $O(n)$.
 $\uparrow$
 [A.V.L Tree]

```

}
public Node getRight() {
    return right;
}
public void setRight(Node right) {
    this.right = right;
}
@Override
public String toString() {
    return "Node [data=" + data + "]";
}
}

```

BST Operations

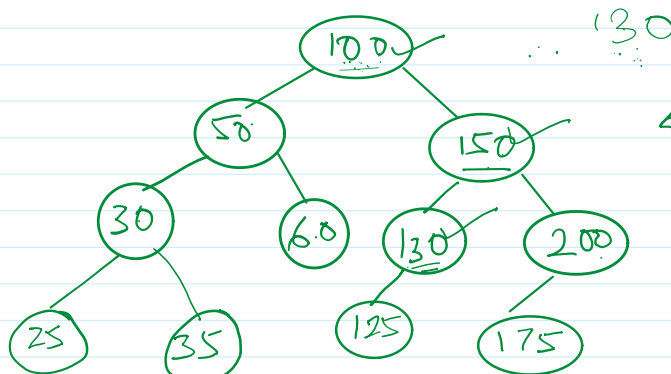
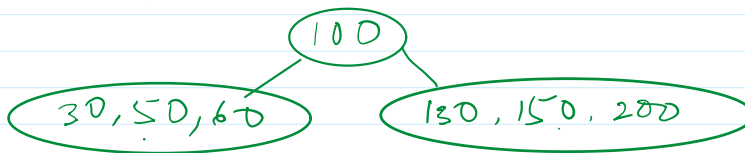
Insertion:

1. If tree is empty -> create root
2. If value is smaller -> move left.
3. If value is larger -> move right.
4. Repeat until NULL is found.

Search operation

Inorder ==> 30 ==> 50 ==> 60 ==> 100 ==> 130 ==> 150 ==> 200

Preorder ==> 100 ==> 50 ==> 30 ==> 60 ==> 150 ==> 130 ==> 200



← Tree is reformed.

we need

Preorder
&
inorder

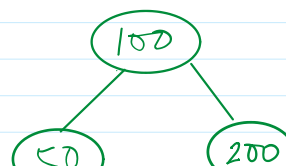
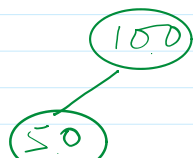
Postorder
&
inorder

Deletion in BST

Deletion is the most important BST operation

Case-1

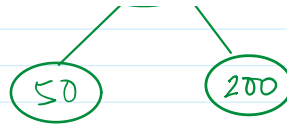
Node is a leaf:



7 100 50

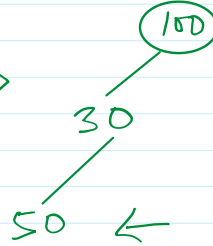
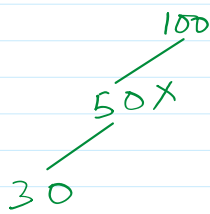


Delete '50' directly



Delete '200' directly.

Node has one child:



← Delete '50' directly

Delete '50'

replace '50' with 30

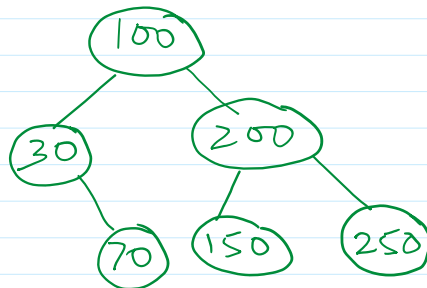
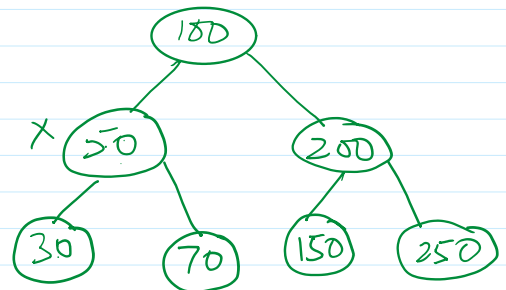
Node has two children:

Delete '50',

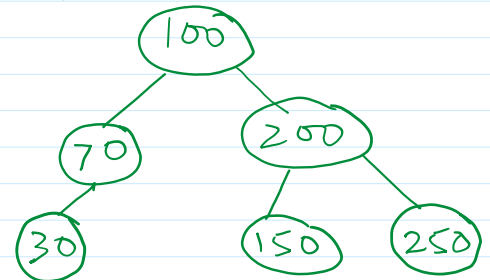
Replace it:

(*) In-order Successor (minimum in right sub-tree)

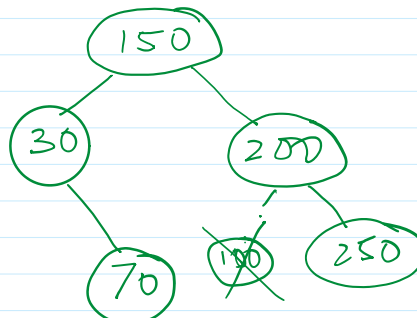
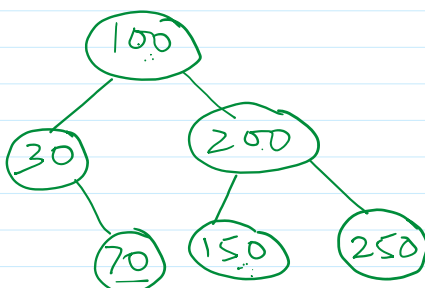
OR
(*) In-order Predecessor (maximum in left sub-tree)



OR

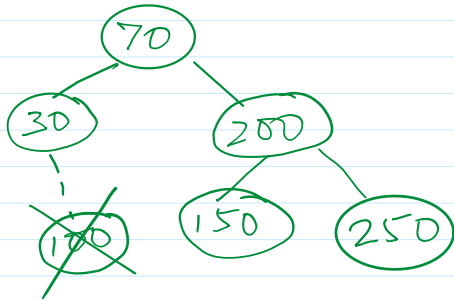


Delete 100. from:



70 150 250

70 ~~100~~ 250



in-order successor

↑
minimum value in right-subtree.

in-order-predecessor
(maximum value in left subtree)

==>20==>30==>40==>50==>55==>60==>70==>90

